

Lucrarea nr. 9: **Gestionarea unei matrice de taste cu organizarea 4x4**

1. Scopul lucrării

În această lucrare se prezintă funcționarea unui sistem logic ceva mai complex, este vorba de un sistem de gestionare a unei matrice de taste cu organizarea 4x4. Pe acest exemplu concret se pot vedea la lucru mai multe tipuri de circuite logice elementare precum: DCD, MUX, numărătoare binare etc.

Lucrarea de față prezentăm o modalitate de gestionare a unei tastaturi cu 16 taste capabilă să recunoască apăsarea unei singure taste, deci insensibilă la combinațiile de taste.

2. Considerente teoretice

2.1. Metode de codificare a unei matrice de taste

Tastatura a fost și probabil va rămâne pentru încă mult timp un mijloc facil prin care se pot introduce comenzi într-un sistem digital, sau informații într-un sistem de calcul. De regulă, pentru fiecare tastă, prin convenție, este asociat un cod binar prin intermediul căreia ea poate fi recunoscută de către sistemul de calcul. În funcție de complexitatea și mărimea tastaturii există o mulțime de soluții de implementare a unei astfel de aplicații.

În cele ce urmează sugerăm câteva idei de implementare pentru cazul unei tastaturi alcătuită din 16 taste, considerând doar situațiile de apăsare a unei singure taste.

Soluția 1

O primă soluție pentru codificarea celor 16 taste ar fi conceperea unui CLC cu 16 intrări (câte una pentru fiecare tastă) și 5 ieșiri (4 pentru codul binar al tastei apăsate și una pentru semnalizarea evenimentului "tastă apăsată"). Se poate concepe schema electrică astfel neapăsarea tastei să mențină intrarea CLC-ului în unu logic, iar apăsarea să aducă respectiva intrare în zero.

Dacă mergem pe această idee, constatăm destul de repede că tabelul de adevăr este foarte mare: avem de implementat 5 funcții binare ce depinde de 16 variabile. Complexitatea sistemului rezultat este prea mare pentru ca această metodă să fie acceptată din punct de vedere practic.

Această metodă prezintă totuși avantajul că permite, prin extinderea numărului de ieșiri și prin alcătuirea corectă a tabelului de adevăr, generarea de coduri binare distincte și pentru situații în care două sau mai multe taste sunt apăsate.

Soluția 2

O altă soluție, ține cont de câteva aspecte practice legate de utilizarea tastaturilor, mai precis de viteza finită de acționare a tastelor:

- durata de apăsare a unei taste este de câteva zeci de milisecunde ($1\text{m}=10^{-3}\text{s}$), chiar dacă ne străduim să facem o apăsare foarte scurtă;
- intervalul de timp dintre două apăsări este de ordinul zecilor de milisecunde chiar și pentru cea mai rapidă secretară;

Ținând cont de aceste aspecte rezultă că nu este nevoie ca circuitul să analizeze în același timp starea logică a tuturor tastelor. Se poate imagina un circuit care să analizeze succesiv starea tastelor: se analizează mai întâi starea primei taste, apoi starea următoarei taste și așa mai departe până se ajunge la ultima tastă după care procesul se repetă la nesfârșit (atâta timp cât circuitul este alimentat). Pentru acest mod de lucru, frecvența de trecere de la o tastă la alta trebuie să fie suficient de mare pentru ca întreaga tastatură să fie verificată cel puțin odată pentru cel mai mic interval de apăsare a unei taste (acesta se poate determina experimental și este dependent și de viteza de reacție a utilizatorului tastaturii).

Schema de principiu a unui astfel de circuit este prezentată în figura 1, pentru cazul unei tastaturi formată din 4 taste. Blocul funcțional denumit "Logică de control", este responsabil de generarea unor coduri binare pe doi biți (Q_1, Q_0), cu următoarea succesiune: ... 00 → 01 → 10 → 11 → 00 → 01 ... Aceste coduri binare sunt folosite pentru comanda intrărilor de selecție ale circuitului MUX 4:1. În acest mod, schimbarea codului binar înseamnă de fapt analizarea stării logice pentru o altă tastă.

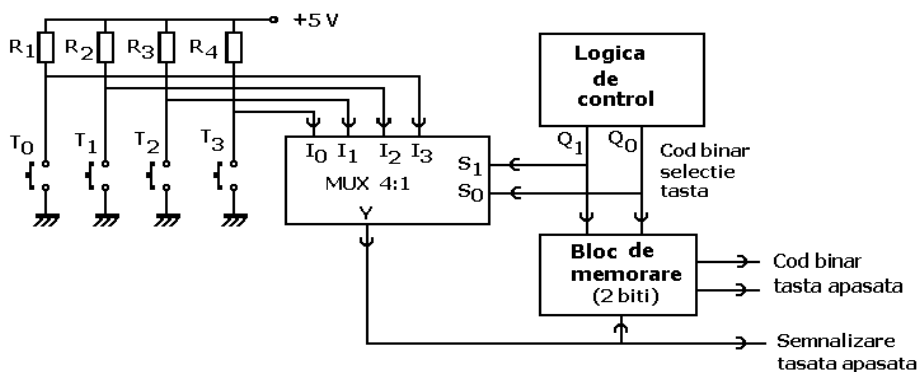


Fig. 1. Schema logică de principiu pentru gestionarea unei tastaturi cu 4 taste

Pentru explicarea funcționării vom considera că $Q_1 Q_0 = 10$. În aceste condiții, la ieșirea Y a multiplexorului vom regăsi starea logică de la intrarea de date I_2 . Datorită schemei electrice, starea logică a intrării de date I_2 este dependentă de starea tastei T_2 :

- dacă T_2 nu este apăsată, intrarea de date I_2 se află conectată la unu logic prin intermediul rezistenței R_3 , în consecință $Y=1$;
- dacă tasta este apăsată, intrarea de date I_2 este conectată la masă prin închiderea contactului T_2 , în consecință $Y=0$;

După ce a expirat timpul alocat combinației $Q_1 Q_0 = 10$, logica de control schimbă codul (se trece la combinația $Q_1 Q_0 = 11$) și astfel se testează starea logică a tastei următoare (în cazul de față, tasta T_3).

Indiferent de codul de selecție generat de logica de control, trecerea în zero logic a ieșirii multiplexorului are semnificația de *tastă apăsată*. În consecință, această ieșire activă pe zero logic, poate fi folosită ca ieșire de semnalizare spre sistemul de calcul. Totodată, trecerea în zero a ieșirii Y mai este folosită și pentru generarea unei comenzi de memorare a codului de selecție al tastei apăsată, în blocul de memorie.

Așa cum este prezentată în figura 1, schema prezintă câteva limitări:

- nu se pot genera coduri pentru apăsarea unor combinații de două sau mai multe taste;
- generează semnalizări multiple pentru o singură apăsare a unei taste;
- mărirea numărului de taste necesită mărirea numărului de intrări ale multiplexorului.

Soluția 3

Pentru un număr mare de taste se folosește tot principiul prezentat anterior cu deosebirea că tastele sunt organizate într-o matrice de m linii și n coloane. Procedând astfel se reduce semnificativ numărul intrărilor de date ale circuitului de multiplexare.

O schema bloc de principiu, pentru cazul unei tastaturi cu 16 taste, se prezintă în figura 2. În primul rând trebuie să remarcăm că 16 taste necesită un cod binar de selecție pe 4 biți. Cei mai semnificativi 2 biți ($Q_3 Q_2$) sunt folosiți pentru comanda intrărilor de selecție ale DCD iar restul de biți pentru comanda intrărilor de selecție ale MUX4:1.

Pentru fiecare tastă este necesar ca un capăt să poată fi conectat la unu logic iar celălalt la masă, cu alte cuvinte un capăt la +Vcc iar celălalt la masă. Organizarea tastelor în matrice este făcută astfel:

- Toate tastele de pe o coloană sunt conectate între ele și sunt conectate printr-o rezistență la +Vcc. Așadar, în orice moment de timp, pe coloane există valoarea logică unu.
- Toate tastele de pe o linie sunt conectate între ele și sunt comandate de către o ieșire (activă pe zero logic) a DCD-ului. Deoarece DCD-ul activează la un moment dat doar o singură ieșire, se poate trage concluzia că există doar o singură linie ce primește zero logic.

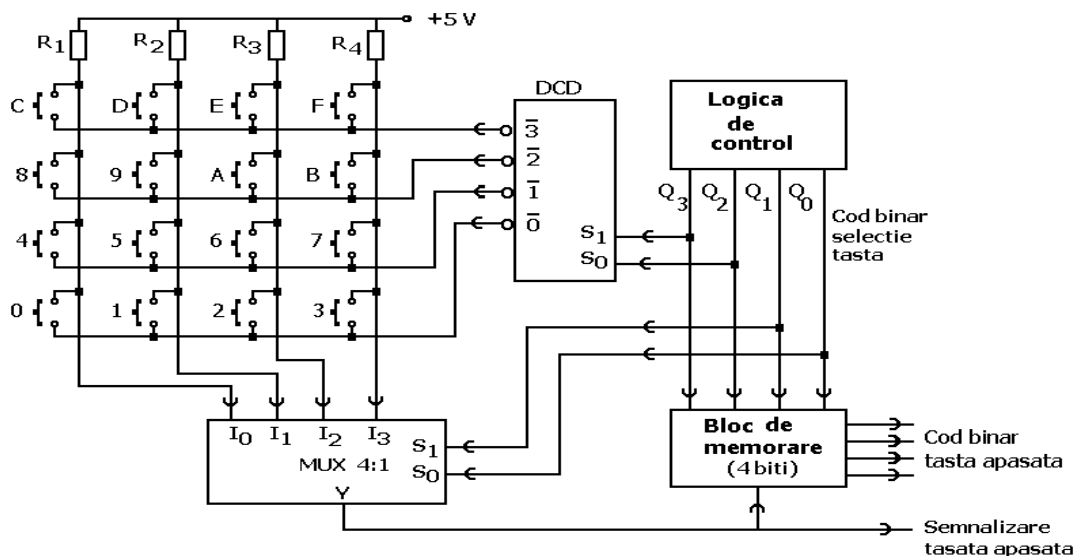


Fig. 2. Schema bloc de principiu pentru gestionarea unei matrice de taste cu organizarea 4 linii X 4 coloane

Pentru explicarea funcționării considerăm situația $Q_3 Q_2 Q_1 Q_0 = 1001$. Facem observația că $Q_3 Q_2$ determină codul liniei (rândului) din matrice iar $Q_1 Q_0$ codul coloanei din matrice.

- Intrarea de selecția a DCD primește codul binar $S_1 S_0 = Q_3 Q_2 = 10$, fapt ce determină activarea ieșirii $\bar{2}$. Așadar, se transmite un zero logic pe linia tastelor **8, 9, A, B**. Tasta verificată efectiv se stabilește prin adresa de coloană dată de biții $Q_1 Q_0$.
- Intrarea de selecția a MUX4:1 primește codul binar $S_1 S_0 = Q_1 Q_0 = 01$, fapt ce determină selecția intrării de date I_1 . Aceasta înseamnă că se analizează starea tastei aflate la intersecția liniei 2 cu coloana 1, este vorba de tasta **9**. Atenție că numerotarea liniilor și a coloanelor începe de la 0!
- Dacă tasta **9** este neapăsată, pe intrarea de date I_1 ajunge un unu logic datorat rezistenței R_2 . Dacă tasta **9** este apăsată, ea face o legătură între ieșirea $\bar{2}$ a DCD și intrarea I_1 . Deoarece ieșirea $\bar{2} = 0$, rezultă că $I_1 = 0$.

- Așadar, pentru codul $Q_3 Q_2 Q_1 Q_0 = 1001$, se verifică starea tastei **9**.
- După expirarea timpului alocat codului $Q_3 Q_2 Q_1 Q_0 = 1001$, logica de control generează codul următor, adică $Q_3 Q_2 Q_1 Q_0 = 1010$. Pentru noul cod, funcționarea este similară cu deosebirea că se verifică starea tastei **A**.
- Ieșirea MUX4:1 poate fi privită ca o ieșire activă pe zero logic ce semnalizează evenimentele de tip "tastă apăsată". Totodată, această ieșire este folosită și pentru încărcarea codului de selecție tastă în blocul de memorie deoarece este vorba chiar de codul tastei apăsate.

Această schemă, așa cum este prezentată în figura 2, prezintă câteva limitări:

- nu poate sesiza apăsarea unor combinații de două sau mai multe taste;
- generează semnalizări multiple pentru o singură apăsare a unei taste;

Problema legată de semnalizarea multiplă a aceleiași taste apăsate este eliminată pe macheta de laborator, (vezi figura 3) printr-o metodă destul de simplă.

2.2. Exemplu de implementare în CPLD

Acest exemplu face o descriere a schemei din figura 1, pentru cazul în care avem 8 taste. Pentru schimbarea codurilor de selecție avem nevoie de un semnal cu frecvența între 10kHz +100kHz. Obținerea acestui semnal se poate face printr-un proces de divizare în frecvență a semnalului dat de oscilatorul de pe macheta de laborator cu CPLD, oscilator ce generează un semnal digital cu frecvența de 25,175MHz. Spre exemplu, dacă alegem o divizare în frecvență cu 2^{10} , se obține un semnal cu frecvența de 24,584kHz.

Observații	Codul VHDL
Secțiune dedicată includerii de librării	-- Exemplu de codificare tastatura library IEEE; use IEEE.std_logic_1164.all;
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: tastatura ; - vector de intrare cu 8 componente ptr. conectare taste: p_taste ; - intrare de ceas: p_clk_in - ieșire ptr. semnalizare cod tastă verificată: p_cod_sel ; - ieșire ptr. semnalizare cod tastă apăsată: p_cod_mem ; - ieșire de semnalizate tastă apăsată: p_signal ;	entity tastatura is port (p_taste:in std_logic_vector(7 downto 0); p_cod_sel:out std_logic_vector(2 downto 0); p_cod_mem:out std_logic_vector(2 downto 0); p_tap: out std_logic; p_clk_in : in std_logic); end tastatura;
Secțiune dedicată descrierii arhitecturii. În cazul de față: - se folosesc trei procese. • Procesul divizare : - este folosit pentru divizarea semnalului de intrare cu frecvența de 25,175MHz. - procesul este sensibil doar la p_clk_in ; - constanta de divizare este 2^{10} deoarece se folosește un numărător pe 10 biți; - codul de selecție tastă este preluat direct de la cei mai importanți trei biți ai numărătorului cnt ; • Procesul selectie : - este folosit pentru selecția tastei a cărei stare trebuie verificată; - procesul este sensibil doar la cod_sel ; - starea logică a tastei este copiată în semnalul test ; - fiecare cod de selecție alege o altă tastă de intrare; • Procesul atrb : - este folosit pentru comanda semnalelor externe (cod selecție, codul ultimei taste apăsate, semnalizare tastă apăsată) ; - procesul este sensibil doar la cod_test ;	architecture arh_tastatura of tastatura is signal cod_sel:std_logic_vector(2 downto 0); signal cod_mem:std_logic_vector(2 downto 0); signal test: std_logic; begin -- proces pentru divizare ceas divizare: process (p_clk_in) variable cnt: std_logic_vector(9 downto 0); begin if p_clk_in'event and p_clk_in='1' cnt := cnt + 1; end if; cod_sel(2 downto 0) <= cnt(9 downto 7); end process divizare; -- proces pentru selecție tasta selectie: process (cod_sel) begin case cod_sel is when "000" => test <= p_taste(0); when "001" => test <= p_taste(1); when "010" => test <= p_taste(2); when "011" => test <= p_taste(3); when "100" => test <= p_taste(4); when "101" => test <= p_taste(5); when "110" => test <= p_taste(6); when "111" => test <= p_taste(7); when others => test <= '0'; end case; end process selectie; atrb: process (cod_sel) begin if test = '1' then cod_mem <= cod_sel;

	<pre> p_t_apasata <="1"; else p_t_apasata <="0"; end if; pin_cod_mem<=cod_mem; pin_cod_sel<=cod_sel; end process atb; end arh_tastatura; </pre>
Fișierul de constrângeri - tastele sunt implementate cu comutatoarele SW1÷SW8; - codul de selecție se afișează pe LED-urile LD1÷LD3; - codul ultimei taste apăsate se afișează pe LED-urile L6÷LD8; - semnalizarea evenimentului tastă apăsată se face pe LD5;	<pre> NET "p_taste<0>" LOC = "P37"; NET "p_taste<1>" LOC = "P40"; NET "p_taste<2>" LOC = "P43"; NET "p_taste<3>" LOC = "P45"; NET "p_taste<4>" LOC = "P47"; NET "p_taste<5>" LOC = "P50"; NET "p_taste<6>" LOC = "P52"; NET "p_taste<7>" LOC = "P54"; NET "p_clk_in" LOC = "P9"; NET "p_cod_sel<2>" LOC = "P62"; NET "p_cod_sel<1>" LOC = "P65"; NET "p_cod_sel<0>" LOC = "P67"; NET "p_cod_mem<2>" LOC = "P75"; NET "p_cod_mem<1>" LOC = "P80"; NET "p_cod_mem<0>" LOC = "P82"; NET "p_t_apasata" LOC = "P71"; </pre>



3. Desfășurarea lucrării

3.1. Studiul machetei cu componente discrete

A. Referitor la schema din figura 2, răspundeți la următoarele întrebări:

- Câte perioade de ceas durează un proces de verificare a tuturor tastelor ?
- Câte perioade de ceas este activă ieșirea de semnalizare "tastă apăsată" ?
- Cum se explică fenomenul de semnalizare multiplă a aceleiași taste apăsate ?
- Ce se întâmplă cu procesul de schimbarea a codurilor pe durata de timp în care tastele sunt apăsate ? Se continuă sau se oprește ? Cum ar fi mai bine ?
- Ce se întâmplă cu semnalul de ieșire dacă apăsarea unei taste este ceva mai lungă ?
- Care este intervalul maxim de timp (exprimat în perioade ale semnalului de ceas) dintre apăsarea propriu-zisă a tastei și activarea ieșirii de semnalizare "tastă apăsată" ?
- Ce cod se generează la apăsarea simultană a două taste ?

B. Referitor la schema din figura 3, macheta cu componente discrete din laborator, răspundeți la următoarele întrebări:

- Identificați circuitele și blocurile funcționale;
- Cum s-a rezolvat problema semnalizării multiple pentru apăsarea aceleiași taste ?
- Explicați de ce liniile matricei de taste sunt comandate de ieșirile 0, 2, 4, 6 ale decodificatorului ? Ce modificări trebuie efectuate în schema pentru a face comanda pe linii cu ieșirile 0, 1, 2, 3 ?
- Ce modificări trebuie efectuate în schema pentru ca informația de pe coloanele matricei de taste să fie preluate pe intrările D4, D5, D6, D7 ? Dar pentru intrările D0, D1, D6, D7 ?
- Care LED semnalizează bitul cel mai puțin semnificativ al codului binar al tastei verificate ?
- Ce rol au inversoarele ? Se poate concepe un bloc de semnalizare optică fără ele ?
- Ce rol are latch-ul din blocul funcțional denumit "ceas manual" ?
- Poarta NAND din oscilator, cu caracteristică de tip trigger Schmitt, poate fi înlocuită cu una cu caracteristică normală ?
- Poarta NAND de la intrarea număratorului poate fi înlocuită cu o poartă cu caracteristică normală ? Câte intrări sunt necesare pentru această poartă ? Care este starea logică de la ieșirea porții atunci când o tastă este menținută apăsată ?
- Prezentați o modalitate de memorare a codului binar al ultimei taste apăsate.

C. Determinări experimentale:

- Plasați comutatorul **K** pe poziția **a** și determinați cu osciloscopul frecvența semnalului de ceas (folosit pentru trecerea de la o tastă la alta); De ce toate LED-urile din blocul de semnalizare luminoasă a codului tastei selectate par în permanență aprinse ?
- Plasați comutatorul **K** pe poziția **b** și acționați asupra tastelor și urmăriți secvența de verificare a acestora; Verificați starea logică de la ieșirea porții prin care trec impulsurile de ceas atunci când o tastă este menținută apăsată ?

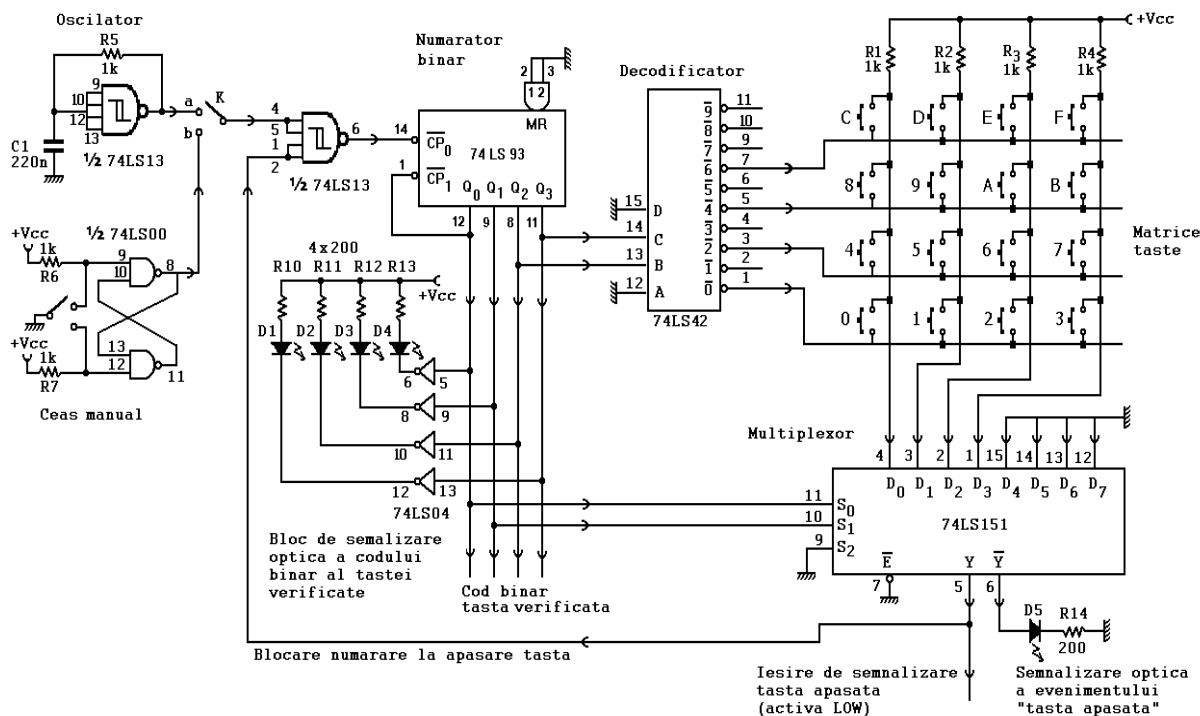


Fig. 3. Schema electrică completă a machetei de laborator reprezentând un sistem logic de gestionare a unei matrice de taste cu organizarea 4x4.

3.2. Studiul pe macheta cu CPLD.

- A.** Verificați codul VHDL din exemplul prezentat în secțiunea 2.2. pe macheta de laborator și răspundeți la următoarele întrebări:
- Cum explicați faptul că LED-urile ce afișează codul tastei verificate par mereu aprinse?
 - Mențineți o tastă apăsată și vizualizați cu osciloscopul catodic semnalul ce comandă LED-ul de semnalizare a evenimentului **tastă apăsată**. Ce observații puteți face în legătură cu numărul de semnalizări ale aceleiași taste apăsată?
- B.** Modificați codul VHDL prezentat în secțiunea 2.2. astfel încât procesul de scanare a restului de taste să fie oprit atunci când o tastă este apăsată.
- C.** Adăugați la proiectul anterior tot ce este necesar pentru a obține afișarea în zecimal (pe unul din afișajele cu 7 segmente) a codului tastei apăsată.
- D.** Folosind cunoștințele acumulate până în prezent realizați o aplicație mai amplă care să emuleze funcționarea unui calculator electronic pentru operația de adunare. Pentru aceasta se pleacă de la următoarele ipoteze:
- Gestionarea multiplexată în timp a tastaturii;
 - Semnificația tastelor este: 0 → SW1, 1 → SW2, 2 → SW3, 3 → SW4, + → SW5, = → SW6, CE → SW7 iar tastele SW8 nu are nici o semnificație;
 - Afișarea se face numai pe un singur digit (acest lucru este posibil deoarece cel mai mare număr este 4, deci rezultatul maxim al adunării este 8).
 - Imediat după pornire se afișează 0 iar după apăsarea unei taste se afișează operandul sau operația sau rezultatul operației. Pentru afișarea operației se va aprinde segmentul **f** al afișajului.

